# B3: It Is Running, but Is It Still Working?

## Monitoring and drift: explainer for presentation and Q&A preparation

94-879 Fundamentals of Operationalizing AI, Track B · Owner: Vidhi Kothari · Team: Lauren Johnson, Anastasiya Joyner, Vidhi Kothari, Stephen Mai · Branch: `B3---Vidhi`

## How to use this document

- Section 1 is the one-page summary. Section 2 states what kind of evidence this project has, and what it does not have.
- Sections 3 to 9 follow the assignment's process: diagnose, hypothesize, intervene, test, evaluate.
- Section 10 covers the recommendation and limitations. Sections 11 and 12 give the decision log with provenance and the AI-supervision notes.
- Section 13 has likely questions; Section 15 is a glossary; Section 16 lists external references.

All empirical results in this document are read directly from the committed outputs in `data/b3_runs/` (`scenarios.json`, `diagnosis.json`, `sensitivity_summary.json`, `exploration.json`) when the PDF is built. Design parameters, thresholds, tool defaults and implementation choices are identified separately by their provenance (Section 6.5).

# 1. Summary

**The assignment's question:** requests succeed and the API is available. Is the AI still behaving appropriately? The task was to design and test monitoring that separates "is the system up?" from "is the AI performing appropriately?", decide which changes matter, and say what should follow an alert.

**Where the hypothesis came from.** The core hypothesis was submitted at the interim checkpoint: infrastructure health and raw feature-drift tests alone are not enough to tell whether the AI is still working appropriately, and feature drift should trigger investigation rather than automatic retraining. That hypothesis survived. What changed was our understanding of the data: the interim treated the split-half drift as a false alarm; later diagnosis showed it was a real model-behavior change caused by the rolling 60 s trade-intensity window refilling after a gap in the recording. We revised the experiment rather than forcing the original explanation to fit (Sections 4.3 and 5).

**Diagnosis.**

- Serving health is good: containers broker running/healthy, mlflow running/healthy, scorer running/healthy; /health reports the model loaded; 0 API errors across all scenarios; p95 latency 2.8-3.1 ms in the saved run.
- Model performance: on the reference window, PR-AUC is 0.070 against a prevalence baseline of 0.073. Above-baseline performance is not established on the available data.
- The model's score correlates 0.80 with **trade intensity** (messages in the last 60 s) and 0.006 with the spike label. Trade intensity has the model's largest weight (+0.507) but correlates -0.033 with the label.
- The supplied drift tool (Evidently, KS test) reports the harmless spread change and the alert-volume change with the same kind of verdict (p close to 0) and no indication of which matters.

**Intervention.** A tiered monitor in one script (`monitoring/b3_monitor.py`): Tier 1 serving status; Tier 2 drift graded by its effect on the model's output and alert volume; a drift severity (OK / LOW / MEDIUM / HIGH, with PAGE for complete serving failure); and Tier 3, a separate retrospective model-performance diagnostic.

**Testing.** Four **controlled development/evaluation scenarios**, S0-S3, plus an internal threshold-robustness analysis.

| Scenario | How it is built | Before (Evidently-style KS) | After (monitor) | Matched intended operational interpretation? |
|---|---|---|---|---|
| S0 | Interleaved odd vs even rows (no change) | 0/4 flagged | OK | Yes |
| S1 | Drifted file (spread wider) | 1/4 flagged | LOW | Yes |
| S2 | Split-half of reference | 3/4 flagged | HIGH (alerts 13.5% to 78.3%) | Yes |
| S3 | Message loss, replayed through Kafka | 2/4 flagged | HIGH (alerts 45.9% to 1.8%) | Yes |

**Conclusion (bounded to this experiment).** Infrastructure health alone does not establish appropriate model behavior. In these controlled scenarios, feature drift by itself could be operationally harmless, while large changes in model outputs and alert volume identified consequential behavior changes. The monitor therefore separates serving health, input/output drift, operational alert impact and retrospective model-performance evidence rather than treating all statistical drift as equivalent. Across 1,485 tested monitoring-setting combinations, the S0-S3 ordering remained unchanged. This suggests that the observed scenario separation is

not dependent on the exact threshold and PSI settings tested. Because all scenarios ultimately derive from the same underlying recording and informed monitor development, this is an internal robustness analysis, not independent validation.

# 2. Methodological status: what kind of evidence this is

The evidence hierarchy for B3:

| Level | What it is | What it can show |
|---|---|---|
| 1. S0-S3 | Controlled development/evaluation scenarios | How the frozen monitor behaves under four known conditions |
| 2. Sensitivity analysis | Internal threshold-robustness analysis | Whether the S0-S3 conclusions depend on the exact threshold and PSI settings tested |
| 3. Held-out evaluation | Not available | Independent generalization remains untested: no genuinely new market recording exists in the supplied repository |
| 4. Remaining uncertainty | | Whether the frozen monitor behaves appropriately on unseen market windows and different failure modes |

**Independent validation limitation.** All supplied market-data artifacts ultimately derive from one approximately five-minute recording, including the repeated training data and transformed scenario files. The monitoring design was also informed by exploratory analysis of this recording. Therefore, S0-S3 should be interpreted as controlled development/evaluation scenarios rather than a held-out test set. The sensitivity analysis shows that their ordering is robust to the tested monitoring settings, but it does not establish generalization to unseen data or failure modes. A future evaluation should freeze the monitor and test it once on an independently recorded replay window.

The final monitor configuration was frozen before the final robustness analysis, but some design choices were informed by earlier exploratory analysis. We therefore do not describe the experiment as preregistered.

# 3. Background: the system and the data

## 3.1 The pipeline

Recorded market messages are replayed into **Kafka**. The **featurizer** computes four features per message and saves them to a **parquet** file. A Flask **API** (the scorer) serves model scores at POST /score, with a health check at GET /health. MLflow stores training runs.

## 3.2 The model and the label

- A **logistic regression** (`models/train_baseline.py`, class_weight = balanced) outputs a probability that a volatility spike is coming.
- The scorer raises a spike alert when that probability is 0.5 or more (supplied, `score_service.py`).
- The **label** is known only afterwards: the standard deviation of returns over the next 20 messages, called a spike if it is at or above the training file's 90th percentile (**tau** = 5.650e-05). Supplied: `label_spikes` defaults. Because the label needs the next 20 messages, labels always arrive late.

Model weights on standardized features:

| Feature | Plain meaning | Weight |
|---|---|---|
| trade_intensity | Messages in the last 60 seconds; low until that 60 s window has filled | +0.507 |
| order_book_imbalance | Whether more quantity waits at the best buy or best sell price (-1 to +1) | -0.471 |
| midprice_returns | Percent change in the middle price since the previous message | -0.064 |
| spread_normalized | Gap between best buy and best sell price, divided by price | +0.028 |

## 3.3 The data: one recording

| File | What it is | Used for |
|---|---|---|
| sample_ticks.jsonl | The only genuine recording: about five minutes, 782 messages, 653 ticker rows | Reference side of S0-S3; diagnosis; exploration; sensitivity |
| sample_ticks_drifted.jsonl | Same messages; only best_ask, best_bid changed (median spread ratio 3.00) | S1 |
| sample_ticks_long.jsonl | The reference repeated 18 times with rewritten timestamps (identical apart from timestamps: True) | Training; tau |
| sample_ticks_msgloss.jsonl | Built by `scripts/b3_inject_faults.py`: every other ticker message removed | S3 |

Because the training file repeats the reference recording, every model number on the reference window is effectively in-sample.

# 4. Diagnosis: symptom and cause

## 4.1 Symptom: healthy service; above-baseline performance not established

| Check | Result (saved run) | Meaning |
|---|---|---|
| Containers | broker running/healthy, mlflow running/healthy, scorer running/healthy | Infrastructure is up |
| Tier 1 status | S0 HEALTHY, S1 HEALTHY, S2 HEALTHY, S3 HEALTHY | Every /score request produced a prediction |
| Latency | p50 1.9-2.0 ms, p95 2.8-3.1 ms | Varies between runs; quoted from the saved run |
| API vs model.pkl | largest score difference 1.1e-16 | The monitor checks the deployed model |
| Training test split | PR-AUC 0.1047; z-score rule 0.0949; prevalence 0.0966 | PR-AUC 0.1047 vs prevalence 0.0966 on the saved training-code split; absolute lift +0.0082 |
| Reference window | PR-AUC 0.070 vs prevalence 0.073; 46 positives in 633 labeled rows | Above-baseline performance not established |
| Alert rate vs spike rate | 45.9% of rows alerted; 7.3% are spikes | Precision at 0.5: 6.4% of alerts are spikes |

## 4.2 Cause: alerts follow trade intensity

| Minute | Rows | Mean trade intensity | Model alert rate | Actual spike rate |
|---|---|---|---|---|
| 0 | 67 | 34 | 0% | 30% |
| 3 | 314 | 158 | 26% | 6% |
| 4 | 249 | 313 | 80% | 2% |
| 5 | 23 | 230 | 91% | 0% |

- The recording has 67 rows (13.6 s), then 171 s with no rows, then 586 rows.
- Trade intensity counts messages in the last 60 s, so it stays low until the window fills. It never exceeds 67 before the gap; after the gap it averages 167 in the first 60 s and 305 afterwards.
- The message rate itself is about the same on both sides of the gap: 4.92 vs 4.74 messages per second.
- Score vs trade intensity r = 0.80; score vs label r = 0.006.
- Sources: `diagnosis.json` (`reference_window`), `minute_table.csv`.

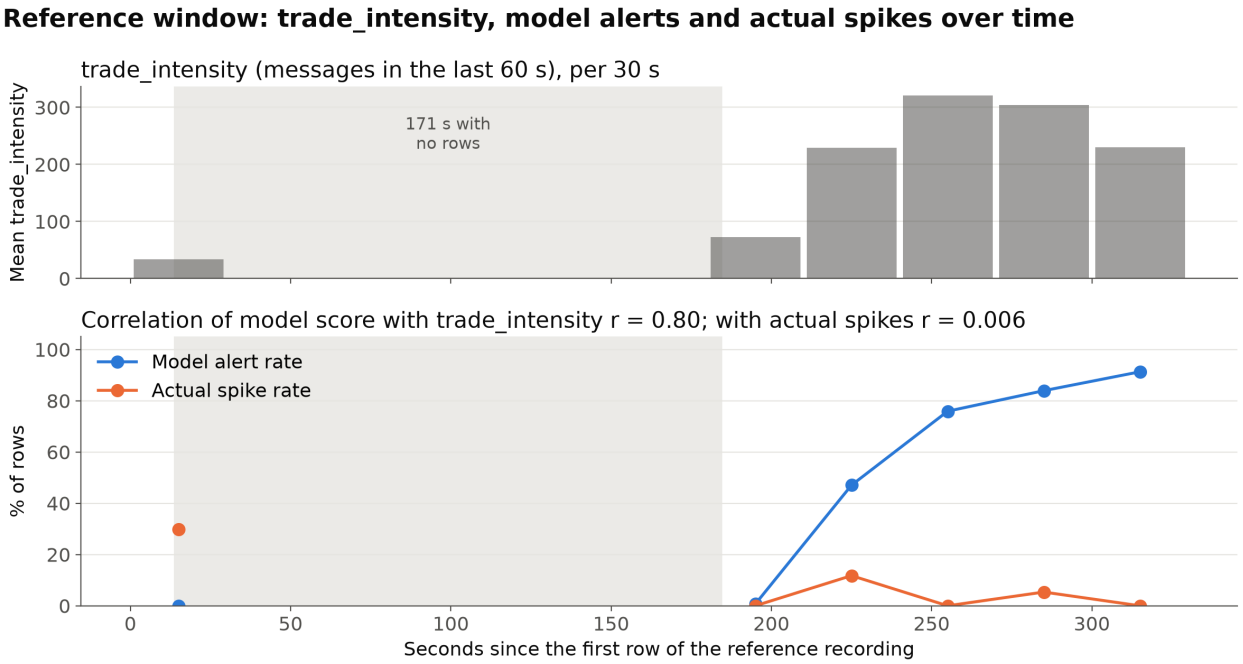



Figure A. Top: mean trade intensity per 30 s; the shaded band is the period with no rows. Bottom: model alert rate (blue) vs actual spike rate (orange).

## 4.3 Correction to the interim: the split-half comparison

**What the interim said.** The interim used the split-half comparison of the reference file (first half vs second half) as an "unchanged" negative control. Evidently flagged 3 of 4 features, and the interim treated those flags as a false alarm, attributing them in part to the featurizer's 60 s trade-intensity window filling over time.

**What later diagnosis showed.** The two halves genuinely differ. The recording has a 171 s gap, and trade intensity is a rolling 60 s count that refills after it. Splitting at the middle row puts the 67 pre-gap rows and 259 refill rows into the first half, so mean trade intensity is 110 vs 303, and the model's alert rate goes from 13.5% to 78.3% while the actual spike rate is 6.1% and 8.5%. The message rate is about the same on both sides of the gap, so the cause is the rolling-window refill mechanism, not a change in message rate.

**How the experiment was revised.** The refill mechanism suggested at the interim was right; "false alarm" was not. The split-half comparison became S2, a real consequential change, and the no-change control was replaced with the interleaved odd/even split (S0), whose two halves cover the same minutes.

# 5. Hypothesis and test setup

## 5.1 The interim hypothesis

Submitted in the interim checkpoint (`B3_interim_checkpoint_v3.md`): a monitoring approach that separates (1) system health, (2) data and prediction drift, and (3) labeled model performance will do a better job of identifying meaningful AI failures than infrastructure checks and per-feature drift tests alone. Feature-level drift should trigger investigation rather than automatic retraining unless it also affects model outputs or performance.

The final monitor is a refinement and operationalization of this hypothesis, not a replacement of it.

## 5.2 What stayed the same from interim to final

- Healthy infrastructure does not imply appropriate AI behavior.
- Raw feature drift alone is not enough to decide operational severity.
- Model-output behavior and labeled model performance are separate monitoring layers.
- Feature drift should trigger investigation, not automatic retraining, unless downstream model behavior or performance is affected.

## 5.3 What changed after the interim

| Interim | Final | Why |
|---|---|---|
| Split-half comparison used as the "unchanged" control; its 3/4 feature flags treated as a false alarm | Split-half is S2, a real consequential change; the no-change control is the interleaved odd/even split (S0) | Diagnosis showed the halves differ because of the gap and the rolling-window refill (Section 4.3) |
| Wording such as "the model is not useful" | "Above-baseline performance is not established on the available data" | The data are short and effectively in-sample, so they cannot show the model is useless |
| A weak-model performance alert was expected because PR-AUC was near prevalence | Tier 3 is a separate retrospective diagnostic with an advisory follow-up action; it never sets drift severity or PAGE status | The weakness is present in every window, including the control; see D7b, D15 |
| General drift alerts and an operational decision of no action / investigate / escalate | Tier 1 serving status; feature drift; score/output drift; alert-volume impact; retrospective labeled performance; severity OK / LOW / MEDIUM / HIGH, with PAGE for serving unavailability | Makes each layer and its action explicit |

## 5.4 The hypothesis as tested in the final run

Infrastructure health checks and per-feature drift tests alone do not tell an operator whether the AI is still behaving appropriately or whether a change matters. A monitor that (1) reports serving status, (2) grades drift by its effect on the model's output and alert volume, and (3) reports retrospective model performance separately will separate no change, harmless change and consequential change in the controlled scenarios.

## What would count against it

- The monitor raises a drift severity (LOW or higher) on the no-change control S0.
- Its ordering does not match the intended operational interpretation, for example S1 HIGH or S3 OK.

- The spread change in S1 materially changes model outputs.

## How the test is set up

- **Intended operational interpretations:** S0 and S3 were built for this project (an interleaved split and an injected message-loss fault); S1 uses the supplied drifted file; S2's interpretation was revised after diagnosis (Section 4.3). Together they give a reference ordering S0=OK, S1=LOW, S2=HIGH, S3=HIGH, set before the final run and sensitivity analysis. This is the intended operational ordering, not ground truth, and it was not defined independently of the exploratory analysis.
- **Same conditions:** the same model file, label definition (fixed tau), reference window and pipeline for every scenario.
- **Before and after on identical inputs:** the Evidently-style KS view and the monitor are computed in the same run on the same rows.
- **Frozen configuration:** the final monitor configuration was frozen before the final robustness analysis, but some design choices were informed by earlier exploratory analysis, so the experiment is not described as preregistered.

# 6. The intervention: tiered monitor

All logic is in `monitoring/b3_monitor.py`. `scripts/b3_inject_faults.py` builds S3; `monitoring/b3_figures.py` draws the figures.

## 6.1 Tier 1: system status (separate from drift severity)

| Status | Meaning | Operational behavior |
|---|---|---|
| HEALTHY | /health ok, model loaded, every attempted /score request produced a prediction | Proceed |
| DEGRADED | /health ok and at least one prediction produced, but one or more /score requests failed | Reported as a serving/reliability issue; not a drift severity; no paging cutoff |
| UNAVAILABLE | /health fails, the model is not loaded, or no /score request produced a prediction | PAGE |
| NOT_CHECKED | Tier 1 skipped (--no_api) | Tier 1 was not evaluated; this does not mean healthy |

API error count, error rate, p50/p95 latency, container status and API-vs-model.pkl score agreement are recorded every run. No partial-error paging threshold is imposed because no production SLO or validated cutoff was supplied.

## 6.2 Tier 2: drift, graded by effect on the model

- **Feature PSI**: how far each input's distribution moved (reference deciles as bins).
- **Score PSI**: the same measure on the model's output. Because the score is produced by the fitted model from all features together, this captures whether input changes propagate into model behavior.
- **Alert-rate shift and ratio**: change in the share of rows the model alerts on (operator workload).
- **KS view ("before")**: per-feature KS p-value flagged below 0.05, dataset drift when 50% or more features are flagged (Evidently defaults). Evidently itself, run on the same pairs, gives the same counts (S0 0/4, S1 1/4, S2 3/4, S3 2/4).

**Why not the KS p-value alone?** The KS p-value indicates evidence of a distributional difference under its assumptions but does not measure operational importance. SciPy's two-sample KS test assumes independent samples. Our streaming rows are serially dependent, so that assumption is not fully satisfied; we therefore do not treat the nominal p-values as calibrated measures of operational consequence. We retain KS as the baseline/before comparison and use effect size and downstream model behavior to assess consequence.

## 6.3 Drift severity (Tier 1 and Tier 2 only)

| Level | Triggered when (first matching rule wins) | Action |
|---|---|---|
| PAGE | Tier 1 UNAVAILABLE | Page on-call; restore the service. Model checks are not trustworthy until it is healthy. |
| HIGH | Alert rate doubles or halves (ratio 2x), or shifts by 20 percentage points or more | Escalate now to model owner and pipeline on-call; consider pausing alerts. Labeled evidence is needed to establish a performance problem; root-cause analysis determines whether retraining is appropriate |

| Level | Triggered when (first matching rule wins) | Action |
|---|---|---|
| MEDIUM | Score PSI 0.10 or more, or alert-rate shift above 5 percentage points | Investigate same day; notify model owner; no retraining unless labeled evidence establishes a performance problem and root-cause analysis supports retraining |
| LOW | Any feature PSI 0.10 or more, with no MEDIUM/HIGH condition | Data-quality ticket: real change or feed bug? No model action |
| OK | None of the above | No action |

Tier 3 does not independently make a scenario HIGH or affect any severity.

## 6.4 Tier 3: retrospective model-performance diagnostic

Once labels exist, Tier 3 reports PR-AUC, the prevalence baseline, lift (PR-AUC minus prevalence) and a 95% exploratory row-bootstrap interval for lift (1000 resamples, seed 0). The evaluation rows are sequentially dependent while the bootstrap resamples individual rows independently, so these intervals are used descriptively rather than as formally calibrated confidence intervals or hypothesis tests. Tier 3 is a separate retrospective diagnostic with an advisory follow-up action; it never sets drift severity or PAGE status. Display label when the interval's lower end is 0 or below: **ABOVE-BASELINE PERFORMANCE NOT ESTABLISHED ON AVAILABLE DATA**. On the available short, effectively in-sample dataset, PR-AUC does not establish performance above the prevalence baseline. This does not prove the model is useless or incapable of performing better on unseen data.

## 6.5 Where each setting comes from

| Setting | Value | Provenance |
|---|---|---|
| Alert threshold | 0.5 | Supplied code (score_service.py) |
| Spike label | 90th percentile, 20-row horizon | Supplied code (label_spikes defaults) |
| Operational PSI threshold | 0.10 | Externally sourced heuristic: conventional PSI band originating in credit-scorecard practice (Lewis 1994, as traced by Potgieter et al. 2026); not a universal statistically calibrated threshold |
| PSI reference band | 0.25 | Same source; descriptive/figure line only; does not independently determine severity |
| PSI bins | 10 | Matches Feature-engine's current default |
| PSI empty-bin handling | 0.0001 | Our monitor uses the same numeric constant, 0.0001, but a different implementation: Feature-engine replaces/adds the constant for empty or zero bins, whereas our psi() floors bin proportions at that minimum. |
| MEDIUM alert shift | 5 percentage points (pp) | AI-proposed project-specific threshold, accepted by owner |
| HIGH alert shift / ratio | 20 percentage points (pp) / 2x | AI-proposed project-specific thresholds, accepted by owner |
| KS p-value; dataset-drift share | 0.05; 50% | Evidently defaults, used for the baseline comparison |
| Bootstrap resamples; seed | 1000; 0 | AI-selected exploratory implementation choices |

# 7. Controlled development/evaluation scenarios

| # | How it is built | Why it is included | Intended operational interpretation |
|---|---|---|---|
| S0 | Odd vs even rows of the reference | Negative control: both halves cover the same minutes | OK |
| S1 | Reference vs supplied drifted file | Large input change in a feature with a small model weight | LOW |
| S2 | First vs second half of the reference | Real change in trade intensity after the recording gap (Section 4.3) | HIGH |
| S3 | Reference vs message-loss file | Constructed upstream fault, replayed through the real pipeline | HIGH |

**S3 is pipeline-faithful:** raw-message loss (every other ticker message removed), replayed through Kafka, features recomputed by the real featurizer, then evaluated by the monitor with the deployed model's scores. It is a constructed fault used to check whether the monitor reacts to a known change; it is not taken from a real incident.

## Exploratory provenance (not evaluation evidence)

`monitoring/b3_exploration.py` reproduces exploratory checks originally run by the AI assistant before parts of the design were proposed. Its outputs are provenance only. E3 in that script is a **feature-space sensitivity probe, not a pipeline experiment**: feature values were edited directly (for example trade intensity x 0.5, alert rate 44.5% to 0.6% on labeled rows) without Kafka or the featurizer. It suggested that the model is sensitive to trade intensity; S3 is the pipeline-faithful follow-up. E1-E5 values are not used as S0-S3 evidence.

# 8. Results on the controlled scenarios

Source: `scenarios.json` (saved run).

| | S0 | S1 | S2 | S3 |
|---|---|---|---|---|
| Rows (reference / current) | 327 / 326 | 653 / 653 | 326 / 327 | 653 / 327 |
| Before: KS features flagged | 0/4 | 1/4 | 3/4 | 2/4 |
| Before: dataset drift (50% or more) | No | No | Yes | Yes |
| Largest feature PSI | 0.019 | 8.245 | 7.894 | 4.885 |
| Score PSI | 0.011 | 0.005 | 4.433 | 2.989 |
| Alert rate ref to cur | 46.2% to 45.7% | 45.9% to 46.2% | 13.5% to 78.3% | 45.9% to 1.8% |
| Tier 1 status | HEALTHY | HEALTHY | HEALTHY | HEALTHY |
| Severity | OK | LOW | HIGH | HIGH |
| Lift, 95% exploratory row-bootstrap interval | [-0.012, +0.032] | [-0.012, +0.015] | [-0.025, +0.006] | [-0.059, +0.005] |
| Tier 3 label | Above-baseline performance not established | Above-baseline performance not established | Above-baseline performance not established | Above-baseline performance not established |

**Before vs after on the same scenarios**

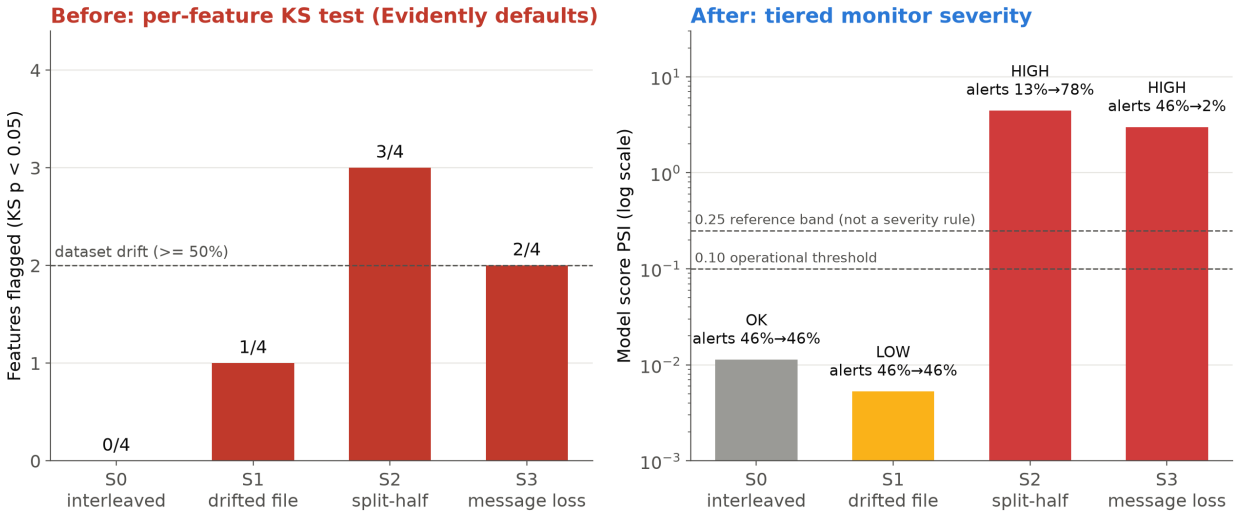



Figure B. Left: features flagged by the KS view. Right: model score PSI (log scale) with the 0.10 operational threshold and 0.25 reference line; bars colored and labeled by severity.

## Reading each scenario

- **S0:** largest feature PSI 0.019, alert rate 46.2% to 45.7%. OK; no drift severity on the control.
- **S1:** spread PSI 8.245 but score PSI 0.005 and alert rate 45.9% to 46.2%. LOW: a data-quality ticket, no retrain. The KS view flags spread with no indication of consequence.
- **S2:** score PSI 4.433, alert rate 13.5% to 78.3%, actual spike rate 6.1% and 8.5%. HIGH.

- **S3:** alert rate 45.9% to 1.8% while Tier 1 is HEALTHY. HIGH. The KS view flags 2 of 4 features, exactly at the 50% dataset-drift line, and says nothing about the alert change.
- **Tier 3:** every exploratory interval includes 0; above-baseline performance is not established on the available data in any window. S3's labels are computed from the reduced message stream.

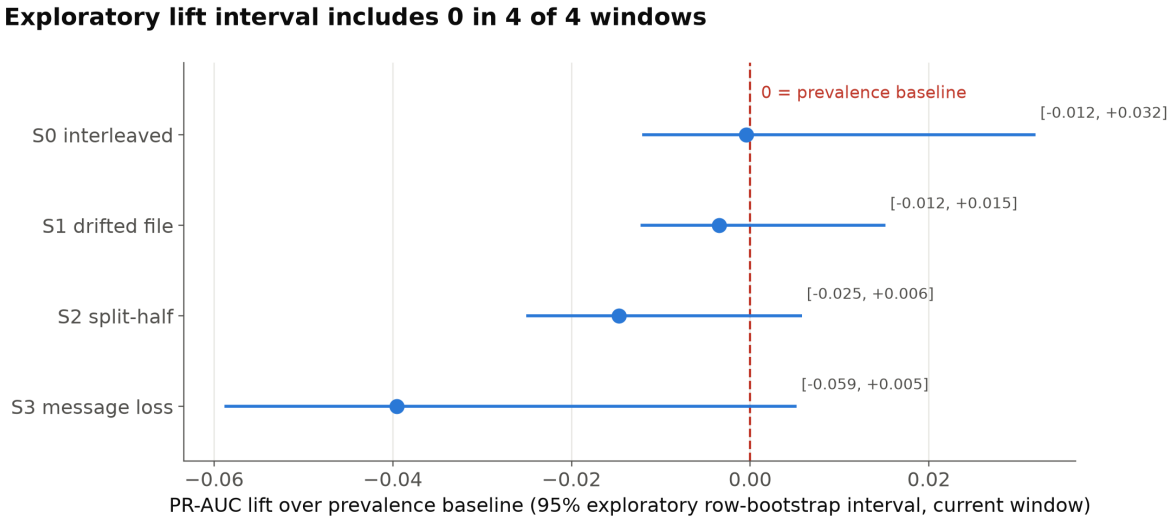



Figure C. PR-AUC lift over the prevalence baseline with the 95% exploratory row-bootstrap interval for each scenario's current window. Descriptive only.

# 9. Internal threshold-robustness analysis

Source: `monitoring/b3_sensitivity.py`, `sensitivity_summary.json`. The existing classify() rules were rerun with alternative settings: operational PSI threshold {0.05, 0.10, 0.15, 0.25, 0.50}; MEDIUM alert shift {2, 5, 10} pp; HIGH alert shift {10, 20, 30, 40} pp; HIGH ratio {1.5, 2, 3}; PSI bins {5, 10, 20}; PSI floor {1e-6, 1e-4, 1e-3}. Combinations with MEDIUM shift at or above HIGH shift were excluded, leaving 1,485. PSI was recomputed for every bins/floor pair. Tier 1 and Tier 3 were not varied.

| Scenario | Matched reference ordering | Severity counts | Largest feature PSI range |
|---|---|---|---|
| S0 | 1485/1485 | OK 1485 | 0.0043-0.0445 |
| S1 | 1485/1485 | LOW 1485 | 5.4992-13.1892 |
| S2 | 1485/1485 | HIGH 1485 | 5.4760-11.8549 |
| S3 | 1485/1485 | HIGH 1485 | 3.2137-7.9899 |

Across 1,485 tested monitoring-setting combinations, the S0-S3 ordering remained unchanged. This suggests that the observed scenario separation is not dependent on the exact threshold and PSI settings tested. Because all scenarios ultimately derive from the same underlying recording and informed monitor development, this is an internal robustness analysis, not independent validation.

**Boundary observation:** S0 was closest to a classification boundary. Its largest feature PSI reached approximately 0.045 under the tested bin/floor settings, versus the lowest tested operational PSI threshold of 0.05. Lower PSI thresholds than those tested would not necessarily preserve S0 = OK.

# 10. Evaluation, recommendation and limitations

## 10.1 Against the hypothesis

| Check | Result | Outcome |
|---|---|---|
| Drift severity on the control S0? | OK | Matched intended interpretation |
| Ordering matches intended interpretation? | OK / LOW / HIGH / HIGH | Matched in the saved run and in all tested settings |
| Does the S1 spread change move the model? | score PSI 0.005, alert rate 45.9% to 46.2% | No material output change |

Within these four controlled scenarios, the monitor matched the intended operational ordering: the no-change control stayed quiet, harmless input drift stayed low-severity, and large output/workload changes escalated. The KS view's dataset-level verdicts (No / No / Yes / Yes) point the same way but carry no severity or action, report the harmless spread change with p close to 0, catch S3 exactly at its 50% line, and say nothing about the model's outputs. Neither drift view nor serving health shows model performance; only Tier 3 does, retrospectively.

## 10.2 Tradeoffs

These are expected tradeoffs reasoned from the design; they were not measured in this project.

- **Fewer drift alarms vs slower detection:** grading by model impact may also quiet small real shifts; LOW tickets are intended to keep them visible.
- **Tier 3 lags:** labels need the next 20 messages, so Tier 2 would be the earlier signal.
- **Settings may need upkeep:** the alert-rate thresholds are project-specific and would likely need revisiting after any retrain.
- **Partial serving failures:** DEGRADED is reported but has no paging cutoff until a real SLO exists.
- **Operating cost:** Tier 1 sends real requests to the API; a busy production service would likely need to sample rows.

## 10.3 Recommendation

- Adopt the tiered monitor (Tier 1 status, drift severity ladder, separate Tier 3 diagnostic) only as a provisional, limited deployment until it has been tested once on an independently recorded replay window.
- Do not retrain because of the spread drift (S1); route it to a data-quality check.
- **Escalate the model for re-evaluation.** Before changing the alert threshold, retraining, or replacing the model, evaluate candidate alternatives on genuinely out-of-sample labeled data.
- Treat breaks and losses in the message stream as an AI risk: in S2 and S3 trade intensity changed, and so did the alerts.

## 10.4 Limitations

**Independent validation limitation.** All supplied market-data artifacts ultimately derive from one approximately five-minute recording, including the repeated training data and transformed scenario files. The monitoring design was also informed by exploratory analysis of this recording. Therefore, S0-S3 should be interpreted as controlled development/evaluation scenarios rather than a held-out test set. The sensitivity analysis shows that their ordering is robust to the tested monitoring

settings, but it does not establish generalization to unseen data or failure modes. A future evaluation should freeze the monitor and test it once on an independently recorded replay window.

- PAGE (UNAVAILABLE) and DEGRADED never occurred in the saved runs. They were checked only by calling the status logic with constructed inputs during development (not saved as a test file), not by stopping or degrading the live scorer.
- Training PR-AUC values come from re-evaluating the saved model with the training code's own split (`diagnosis.json`).
- S3's labels are computed from the reduced message stream.

## 10.5 Remaining uncertainty

Whether the frozen monitor behaves appropriately on unseen market windows and different failure modes, and whether the model performs above the prevalence baseline on independent data.

## 10.6 Team integration points

These are integration points with teammates' challenges, as described in the team's materials; B3 did not independently verify B1 or B2 work. B2 concerns /health reporting model_loaded; B3 adds that a loaded model is not necessarily behaving appropriately. B1 concerns keeping up with the feed; in B3's scenarios, breaks or losses in the message stream changed trade intensity, which drives the model's predictions.

# 11. Decision log with provenance

Full version: `B3_AI_DECISIONS.md`.

| # | Decision | Chosen | Provenance |
|---|---|---|---|
| D1 | Scenarios | S0-S3, S3 replayed through Kafka | Options AI-proposed; owner chose against the AI recommendation (S0-S2 only) |
| D2 | No-change control | Interleaved split | AI-proposed, owner-accepted |
| D3 | Where to split S2 | Middle row | AI-proposed, owner-accepted |
| D4 | Tier 1 measurement | /health + real /score API | AI-proposed, owner-accepted |
| D5 | Drift metric | PSI, with KS as the before view | AI-proposed, owner-accepted |
| D6 | Threshold setting | Fixed values in code; not preregistered | AI-proposed, owner-accepted |
| D7 | Severity structure | Graded by downstream impact | Owner-modified (owner wrote the graded version) |
| D7a | HIGH thresholds | 2x / 20 pp; score PSI alone never HIGH | Numbers AI-proposed; PSI rule owner-added |
| D7b | Model weakness | Separate Tier 3 diagnostic | AI-proposed, owner-accepted |
| D8 | Tier 3 label rule | Interval lower end 0 or below | AI-proposed, owner-accepted; later non-severity (D15) |
| D9 | MEDIUM threshold | 5 pp or score PSI 0.10 | 5 pp AI-proposed, owner-accepted |
| D10-D12 | Interim correction; figures; slides | Visible correction; A+B+C; 4 slides + backup | AI-proposed, owner-accepted |
| D13 | PSI provenance | Lewis (1994) via Potgieter et al. (2026); 0.10 operational, 0.25 reference | Owner-directed |
| D14 | Exploration in code | b3_exploration.py, provenance only | Owner-directed |
| D15 | Tier 3 statistics | Exploratory row-bootstrap interval; no severity role | Owner-directed |
| D16 | PAGE / Tier 1 status | No percentage cutoff; HEALTHY / DEGRADED / UNAVAILABLE / NOT_CHECKED | Owner-directed |
| D17 | Sensitivity analysis | 1,485 combinations | Grid owner-specified; script by AI; run by owner |
| D18 | Held-out data | None available; limitation stated; methodology frozen | Owner-directed |

# 12. AI supervision

**Scope of AI contributions.** The interim checkpoint already established the high-level hypothesis that infrastructure health and raw drift alone were insufficient (Section 5.1); the AI assistant did not originate the B3 research question or hypothesis. AI assistance contributed specific causal explanations and monitor-design choices (thresholds, PSI, the PAGE rule, bootstrap interpretation, the sensitivity-analysis script and other details), each accepted, modified or rejected below. One consequential AI explanation about the split-half drift was revised and then rejected after measurement.

### *1. Rejected after measurement: the AI's explanation of the split-half drift*

**Proposed, in order:** (a) AI assistance contributed the causal interpretation that the split-half flags came in part from the 60-second rolling feature window (the interim states that "part of this comes from the featurizer itself" and treated the split-half as an unchanged control). Subsequent diagnosis showed the rolling-window mechanism was real, but the conclusion that the comparison was a false alarm was wrong; (b) after a check that removed only the first 60 s of the recording, "the message rate roughly triples after a 171 s gap" (in an earlier README, PDF and code descriptions). **Checked:** b3_diagnosis.py measured 4.92 vs 4.74 messages per second on either side of the gap, so (b) was wrong; trade intensity of 167 in the first 60 s after the gap vs 305 afterwards shows the rolling window refilling, so the mechanism in (a) was right but its conclusion was not: the alert rate really changes from 13.5% to 78.3%. **Outcome:** (b) removed everywhere; S2 kept as a real change with the corrected cause.

### *2. Accepted provisionally, then stress-tested: the operational thresholds*

**Proposed:** candidate alert-rate thresholds (5 pp for MEDIUM; 2x or 20 pp for HIGH) and prediction drift alone deciding HIGH. **Response:** the owner replaced "prediction drift decides" with the graded structure and kept score PSI alone at MEDIUM; the numbers were accepted as project-specific settings, not standards. **Checked:** the frozen monitor was rerun across 1,485 setting combinations and the S0-S3 ordering remained unchanged. This tests dependence on parameter choices; it does not independently validate monitor generalization.

### *Other AI recommendations*

- Fix tau from training data: accepted (positive rate equal across reference and drifted windows).
- Unsourced "rule of thumb" wording for PSI 0.10/0.25: rejected; replaced with a cited source.
- Calling the thresholds preregistered: rejected; exploration had preceded the thresholds.
- Paging on a fixed percentage of API errors (set without asking the owner): rejected; replaced by the Tier 1 status design.
- Bootstrap output presented as formal intervals with a degradation rule: modified; relabeled exploratory and removed from severity.
- Recommending a fallback to the z-score rule: rejected; that rule also scored below prevalence.
- Nonstandard labels ("change alerts"): rejected.

# 13. Likely questions

**Q: Why not retrain whenever drift is detected?**

Not all drift matters. S1 has a large input shift (spread PSI 8.245) with score PSI 0.005 and almost no alert change. Labeled evidence is needed to establish a performance problem, and root-cause analysis determines whether retraining is appropriate.

**Q: Your interim said the split-half was a false alarm. Was the interim wrong?**

The core hypothesis survived; the interpretation of that comparison did not. The refill mechanism was right, but the model's alerts really changed (13.5% to 78.3%), so it was not a false alarm. We made it S2 and replaced the control with the interleaved split.

**Q: Why PSI rather than the KS p-value?**

The KS p-value indicates evidence of a difference under its assumptions but not its operational importance; it depends on sample size, and our rows are serially dependent. PSI measures the size of a shift, and score PSI and alert rate show whether the shift reaches the model's behavior. KS is kept as the before view.

**Q: Where do 0.10 and 0.25 come from? Did you tune them?**

They are conventional PSI heuristic bands from credit-scorecard practice (Lewis 1994, as traced by Potgieter et al. 2026), not calibrated thresholds. Only 0.10 is used for severity. The alert-rate thresholds were AI-proposed project settings. Some design choices followed exploratory analysis, so we do not call the experiment preregistered; instead we tested 1,485 setting combinations and the ordering did not change.

**Q: Is S0-S3 a test on unseen data?**

No. All scenarios derive from one five-minute recording that also informed the design. They are controlled development/evaluation scenarios; generalization to unseen data is untested.

**Q: Isn't S3 synthetic?**

It is a constructed fault, built from the supplied recording and replayed through Kafka and the real featurizer. S2 comes from the supplied data directly.

**Q: Does the model work?**

On the available short, effectively in-sample dataset, PR-AUC does not establish performance above the prevalence baseline. This does not prove the model is useless or incapable of performing better on unseen data.

**Q: Why is the alert rate so high?**

The model alerts on 45.9% of reference rows, mostly when trade intensity is high. The supplied logistic regression uses class_weight='balanced', which assigns class weights inversely proportional to class frequency during fitting. We have not tested whether this setting caused the observed alert rate.

**Q: What happens if the scorer fails some requests?**

Tier 1 reports DEGRADED with the exact error count and rate; if no request produces a prediction, it is UNAVAILABLE and pages. No partial-error paging cutoff was set because no SLO was supplied.

**Q: What would you monitor after deploying?**

Tier 1 status and telemetry, score PSI and alert rate per window, Tier 3 once labels arrive, and breaks in the data, since trade intensity drives the score.

**Q: How does B3 relate to B1 and B2?**

As team integration points (not independently verified by B3): B2 concerns whether the model is loaded; B3 checks its behavior. B1 concerns keeping up with the feed; in B3's scenarios, breaks or losses in the message stream changed trade intensity and the model's predictions.

# 14. Reproducing the results

Full commands, including building the feature files through Kafka, are in `B3_README.md`. With the feature files and model in place (Docker needed only for the monitor's Tier 1):

```
python monitoring/b3_monitor.py
python monitoring/b3_diagnosis.py
python monitoring/b3_sensitivity.py
python monitoring/b3_exploration.py
python monitoring/b3_figures.py
```

# 15. Glossary

| Term | Meaning |
|---|---|
| Alert rate | Share of rows where the score is 0.5 or more; stands in for operator workload. |
| Autocorrelation / serial dependence | Each row resembles the rows just before it, as in streaming data. |
| Bid / ask / spread | Highest buyer price, lowest seller price, and the gap between them. |
| Bootstrap (row) | Resampling rows with replacement many times and recomputing a number, to see how much it varies. |
| Controlled development/evaluation scenario | A constructed comparison with an intended interpretation, used to see how the monitor behaves; not a test on unseen data. |
| Correlation (r) | How strongly two numbers move together, from -1 to +1. |
| Data drift | A change in the distribution of a model's inputs. |
| DEGRADED / UNAVAILABLE / NOT_CHECKED | Tier 1 statuses: partial /score failures; no predictions possible (PAGE); Tier 1 not evaluated. |
| Evidently | The supplied drift-report tool; on this data it uses the KS test per feature. |
| Exploratory row-bootstrap interval | The 2.5th-97.5th percentiles of bootstrap results; used descriptively, not as a formal interval estimate. |
| Fault injection | Deliberately introducing a known failure to see whether monitoring responds. |
| In-sample | Evaluated on data used to train the model. Here the reference recording is effectively in-sample because the training file repeats it. S1 and S3 are transformations of the same underlying recording, so none of the scenarios provides genuinely independent out-of-sample data. |
| Internal threshold-robustness analysis | Rerunning the frozen rules with alternative settings on the same scenarios. |
| Kafka / topic / replay | A message queue; a named stream; publishing recorded messages as if live. |
| KS test and p-value | Tests whether two samples differ in distribution; the p-value is evidence of a difference under the test's assumptions, not its importance. |
| Label / spike / tau | The true answer known afterwards; a spike if next-20-message variability is at or above tau. |
| Lift | PR-AUC minus the prevalence baseline. |
| Prediction drift | A change in the distribution of the model's outputs. |
| Prevalence baseline | The PR-AUC a no-information ranking gets: the share of rows that are spikes. |
| PR-AUC | Area under the precision-recall curve: how well scores rank spikes above non-spikes. |
| PSI | Population Stability Index: how much a distribution moved across bins taken from the reference. |
| Score PSI | PSI on the model's output scores. |
| Trade intensity | Messages in the last 60 s; low until the window has filled, e.g. at the start or after a break. |

# 16. External references


1. Potgieter, C. J., Van Zyl, C., Schutte, W. D., et al. (2026). The population resemblance statistic: a chi-square measure of fit for banking. *Annals of Operations Research*, 361, 413-435. https://doi.org/10.1007/s10479-025-07024-6

   Used for: tracing PSI to Lewis (1994); the 0.10 and 0.25 "Lewis constants" (below 0.10 little or no action, 0.10-0.25 investigate, above 0.25 consider reconstruction); and the point that these fixed constants do not account for shift magnitude or sampling variability and are not universal, statistically calibrated thresholds.

2. Lewis (1994), cited only via Potgieter et al. (2026); not consulted directly.

3. Feature-engine, DropHighPSIFeatures source: https://github.com/feature-engine/feature_engine/blob/main/feature_engine/selection/drop_psi_features.py

   Used for: defaults threshold=0.25, bins=10, strategy="equal_frequency", min_pct_empty_bins=0.0001; documentation naming 0.25 and 0.10 as commonly used PSI thresholds; the small constant for empty/zero bins. Our psi() uses the same constant with a different implementation (Section 6.5).

4. SciPy documentation, scipy.stats.ks_2samp: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html

   Used for: the two-sample KS test compares the continuous distributions of two independent samples (Section 6.2).

5. scikit-learn documentation, sklearn.linear_model.LogisticRegression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

   Used for: class_weight='balanced' assigns class weights inversely proportional to class frequency.